Laporan Tugas Kecil 1 IF2211 Strategi Algoritma

Semester II Tahun 2025/2026

Penyelesaian Permainan Queens LinkedIn

Oleh:

Nama : Muhammad Nafis Habibi

NIM : 13524018

Sekolah Teknik Elektro dan Informatika - Institut Teknologi Bandung

Jl. Ganesha 10, Bandung 40132

2026

Daftar Isi

Daftar Isi 1
Deskripsi Tugas 2
Penjelasan Algoritma 2
A. Pencarian Solusi 2
B. Validasi Input 5
Pembuatan Program 6
A. Struktur Program 6
B. GUI 6
C. Visualisasi 7
D. Hasil Kodingan 8
1. App.java 8
2. IndexController.java 9
3. Solver.java 20
4. Validation.java 25
Hasil Pengujian 27
A. Kasus uji 1 27
B. Kasus uji 2 28
C. Kasus uji 3 29
D. Kasus uji 4 30
E. Kasus uji 5 31
Penutup 33
Lampiran 34

Deskripsi Tugas

Queens adalah gim logika yang tersedia pada situs jejaring profesional LinkedIn. Tujuan dari gim ini adalah menempatkan *queen* pada sebuah papan persegi berwarna sehingga terdapat **tepat** hanya satu *queen* pada tiap baris, kolom, dan daerah warna. Selain itu, satu *queen* tidak dapat ditempatkan bersebelahan dengan *queen* lainnya, termasuk secara diagonal.

Penyelesaian harus dilakukan dengan algoritma ***brute force***. Berdasarkan spesifikasi tugas yang diberikan, program harus dapat:

1. Menemukan satu solusi penempatan *queen* pada suatu papan berwarna yang diberikan, atau menampilkan bahwa tidak ada solusi yang valid.
2. Melakukan validasi input yang diberikan.
3. Menampilkan banyak konfigurasi atau iterasi yang ditinjau oleh algoritma
4. Menampilkan waktu eksekusi program dalam *millisecond* (selain waktu *input/output*).
5. Memvisualisasikan proses *brute force* yang dilakukan (Live Update)

Ada beberapa batasan masukan yang perlu divalidasi:

1. Masukan berupa matriks karakter (array dua dimensi) berukuran n x n. Artinya besar baris dan kolom sama.
2. Masukan berupa karakter dari ‘A’ sampai ‘Z’.
3. Besar papan memiliki besar maksimal yaitu 26.
4. Jumlah daerah warna yang ada sama dengan besar papan.
5. Tidak ada daerah warna yang terpisah dari daerah warna yang sama. Daerah warna terhubung secara ortogonal dan tidak diagonal. (Batasan ini sepertinya tidak wajib divalidasi karena sulit untuk di-*brute force*)

Penjelasan Algoritma

A. Pencarian Solusi

Pencarian solusi dilakukan dengan cara menaruh satu *queen* pada tiap baris. Saat mencoba menaruh queen pada suatu kolom, dilakukan pengecekan apakah kolom tersebut sudah diisi oleh *queen* lain, apakah ada *queen* lain pada daerah warna yang sama, dan apakah ada *queen* lain yang bersebelahan secara diagonal (tidak perlu mengecek lagi secara ortogonal). Karena perlu menaruh *queen* pada tiap baris dengan ukuran papan yang tidak menentu, algoritma *brute force* yang dilakukan dibuat dengan pendekatan rekursi untuk mengecek semua permutasi.

Pencatatan kolom mana saja yang sudah ada *queen* dilakukan dengan menggunakan array integer berukuran n yang masing-masing indeks menandakan baris apa yang memakai kolom dengan indeks tersebut. Jika belum ada baris yang memakainya, kekosongan kolom tersebut ditandai dengan nilai -1. Pengecekan apakah ada *queen* yang berdekatan secara

diagonal dapat dilakukan dengan mengecek apakah kolom sebelum atau sesudahnya dipakai oleh baris sebelumnya.

Pencatatan warna apa saja yang sudah ada *queen* dilakukan dengan menggunakan array boolean berukuran 26. Indeks 0 ditujukan untuk karakter 'A', indeks 1 ditujukan untuk karakter 'B', dan seterusnya hingga indeks 25 untuk karakter 'Z'.

Total iterasi dihitung dari berapa banyak iterasi kolom yang dilakukan. Total kasus yang dicek dihitung dari berapa kali kasus tidak valid ditambah dengan satu kasus valid yang ditemui. Total waktu eksekusi dihitung dari inisialisasi variabel hingga selesai rekursi. Visualisasi algoritma dapat dilakukan saat menaruh atau membuang *queen* pada papan.

Solusi ditemukan jika berhasil menaruh *queen* terakhir di baris terakhir. Program akan lanjut iterasi dan rekursi jika solusi belum ditemukan. Jika permutasi terakhir sudah dicek dan solusi tidak ditemukan, program akan mengembalikan papan null. Saat solusi ditemukan, papan solusi bisa dibuat dari informasi pada array integer yang digunakan untuk mencatat pada kolom ini, baris apa yang memakainya.

Pseudocode dari algoritma pencarian solusi adalah sebagai berikut (berdasarkan Java):

```
int totalIteration;
int totalCase;
long executionTime;

char[][] solve(char[][] input) {
    long startTime = now();
    totalIteration = 0;
    totalCase = 0;
    int n = input.length;
    boolean[] usedColor = new boolean[26];
    int[] usedColumn = new int[n];
    for (int i = 0; i < n; i++) {
        usedColor[i] = false;
        usedColumn[i] = -1; // Belum dipakai
    }
    char[][] solution = solveRecursive(0, n, input, new boolean[26],
new int[n]);
    executionTime = now() - startTime;
    return solution;
}

char[][] solveRecursive(int i, int n, char[][] input, boolean[]
usedColor, int[] usedColumn) {
    for (int j = 0; j < n; j++) {
        totalIteration++;
        if (!isPenaruhanValid(i, j, n, input, usedColor,
usedColumn)){
            totalCase++;
            visualize();
        }
        else {
            // Tandai penaruhan
            usedColor[input[i][j] - 'A'] = true; // Warna sudah
dipakai
            usedColumn[j] = i; // Kolom dipakai baris ke-i
```

```
            visualize();

            // Sudah sampai ujung, pasti ini solusinya
            if (i == n - 1) {
                totalCase++;
                return generateGrid(n, input, usedColumn);
            }
            else {
                char[][] solution = solveRecursive(i + 1, n));
                if (solution != null) {
                    return solution;
                }
                else {
                    // Balikkan state
                    usedColor[input[i][j] - 'A'] = false;
                    usedColumn[j] = -1; // Belum dipakai
                    visualize();
                }
            }
        }
        else
    }
    return null;
}
```

Pseudocode dari algoritma pengecekan kevalidan dan juga membuat papan solusi:

```
boolean isPenaruhanValid(int i, int j, int n, char[][] input,
boolean[] usedColor, int[] usedColumn) {
    // Cek kolom
    if (usedColumn[j] != -1) return false;
    // Cek diagonal
    if (i > 0) {
        if (j > 0 && usedColumn[j-1] == i-1) return false;
        if (j < n - 1 && usedColumn[j+1] == i-1) return false;
    }
    // Cek daerah warna
    if (usedColor[input[i][j] - 'A']) return false;

    return true;
}

char[][] generateGrid(int n, char[][] input, int[] usedColumn) {
    char[][] solutionGrid = new char[n][n];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (usedColumn[j] == i) {
                solutionGrid[i][j] = '#';
            } else {
                solutionGrid[i][j] = input[i][j];
            }
        }
    }
    return solutionGrid;
}
```

Kompleksitas waktu algoritma dari pencarian solusi di atas adalah $O(n^n)$. Didapat dari n kemungkinan penaruhan *queen* pada tiap baris yang berjumlah n. Namun, karena adanya pengujian kevalidan di tengah pencarian solusi, kompleksitas waktu algoritma mungkin lebih dekat dengan O(n!). Dari beberapa kasus uji yang dicoba, kompleksitas waktu akhir mungkin lebih kecil dari itu.

B. Validasi Input

Validasi input dimulai dari mengambil array of string. Jika ada panjang string dalam array yang tidak sama dengan panjang array tersebut, berarti tidak valid. Jika panjang array of string sama dengan nol atau lebih dari 26, dihitung juga sebagai tidak valid. Selanjutnya, setiap karakter pada string dicek satu per satu. Jika ada yang memiliki kode warna selain dari 'A' sampai 'Z', input dianggap tidak valid. Warna berbeda yang muncul juga dihitung. Jika banyak warna berbeda yang muncul tidak sama dengan besar papan, input dianggap tidak valid.

Terakhir, algoritma *flood fill* digunakan untuk mencari daerah warna yang terpisah. Pengecekan daerah warna yang terpisah cukup sulit dengan algoritma *brute force*. Karena sudah tidak diwajibkan dan sudah saya buat, validasi ini saya biarkan saja ada.

Adapun *pseudocode* untuk validasi input adalah sebagai berikut:

```
boolean validateInput(char[][] input) {
    int n = input.length;
    for (char[] cs : input) {
        if (cs.length != n) return false;
    }
    if (n == 0 || n > 26) return false;
    int totalRegion = 0;
    boolean[] hasColor = new boolean[26];
    boolean[][] visited = new boolean[n][n];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (input[i][j] < 'A' || input[i][j] > 'Z') return false;
            if (hasColor[input[i][j] - 'A']) {
                if (!visited[i][j]) return false;
            } else {
                hasColor[input[i][j] - 'A'] = true;
                floodFill(i, j, visited, input);
                totalRegion++;
            }
        }
    }
    if (totalRegion != n) return false;

    return true;
}
```

Pembuatan Program

A. Struktur Program

Program dibuat dalam bentuk aplikasi dengan menggunakan bahasa pemrograman Java. Graphical User Interface (GUI) dibuat dengan JavaFX.

Struktur folder dan file utama adalah sebagai berikut:

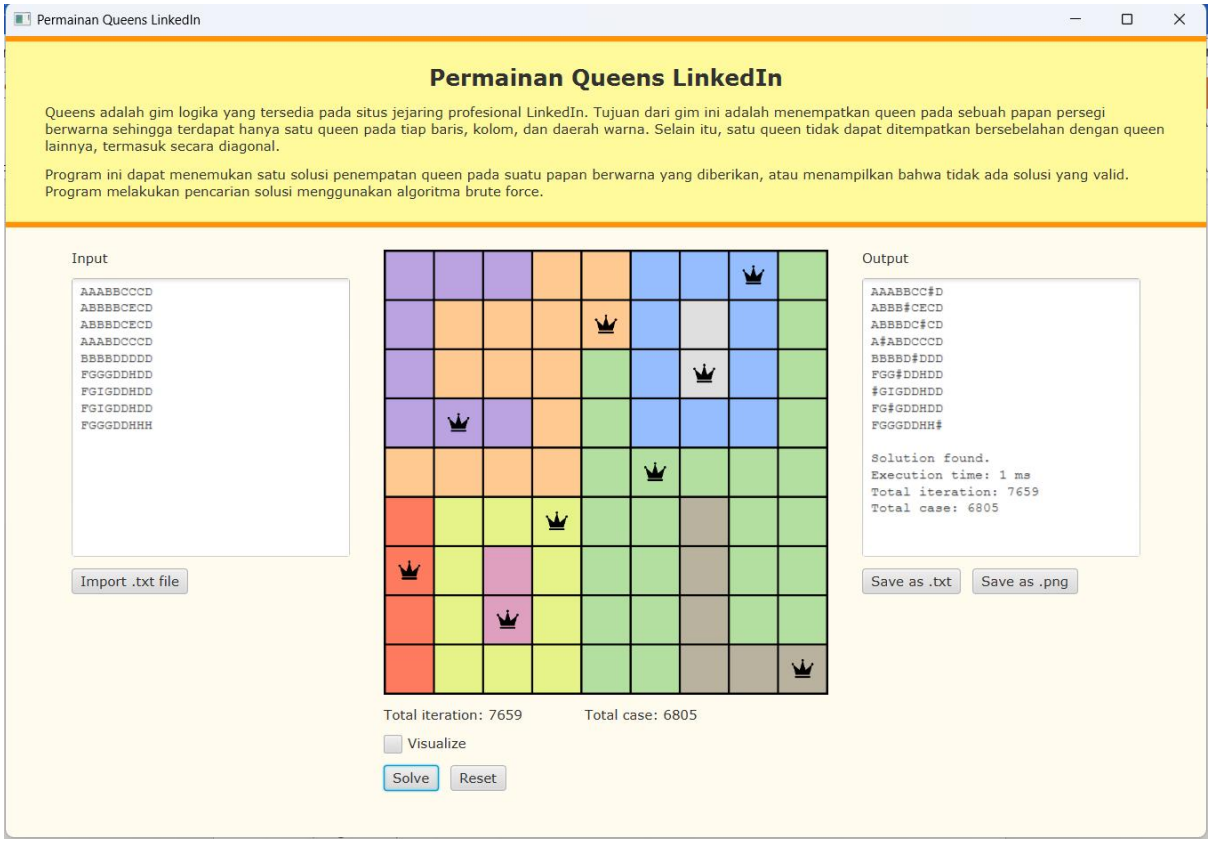
1. bin/ : berisi executable .jar dari hasil *compile* Jawa
2. docs/ : berisi dokumen spesifikasi tugas dan laporan
3. src/ : berisi seluruh file kodingan dan resouces lainnya yang dibutuhkan di aplikasi
4. target/ : berisi copy dari folder src/ dan beberapa hal tambahan dari compiler.
5. test/ : berisi input dan output untuk testing
6. Makefile : untuk menjalankan program
7. README.md : berisi deskripsi program dan instruksi untuk menjalankannya

Terdapat empat kelas utama pada program ini, yaitu:

1. App : Menjalankan aplikasi
2. IndexController : Controller dari halaman utama (index.fxml)
3. Solver : Mencari solusi dari permainan *Queens*
4. Validation : Memvalidasi input yang diberikan

B. GUI

GUI dibuat di file index.fxml dengan controller IndexController.java. Tampilan aplikasi dibagi menjadi empat bagian: deskripsi program, input, visualisasi, dan output.



Input dapat dilakukan dengan mengedit area teks di bagian kiri atau import dari file .txt. Setiap kali input teks berubah, secara otomatis, visualisasi juga ikut berubah jika dan hanya jika inputnya valid. Output berupa teks akan muncul di bagian kanan. Output berupa gambar dapat dilihat dari visualisasi di tengah. Kedua output masing-masing dapat disimpan menjadi .txt dan .png. Error dari input akan muncul di bawah tempat input dan error dari output akan muncul di bawah tempat output.

Setelah memastikan input valid, tombol "Solve" dapat ditekan. Terdapat pilihan untuk menampilkan visualisasi algoritma *brute force* yang dilakukan. Saat melakukan visualisasi algoritma, semua tombol tidak dapat ditekan sampai visualisasi selesai. Tombol "Reset" dapat ditekan untuk menghilangkan semua *queen* yang telah diletakkan setelah melakukan "Solve".

C. Visualisasi

Visualisasi diletakkan di dalam canvas dari JavaFX. Setiap kali input berubah dan valid, isi dari canvas juga berubah sesuai *input* yang diberikan. Setiap petak digambar secara manual (dengan kode).

Untuk membuat visualisasi yang dapat secara *live* melakukan *update*, *threading* perlu dilakukan supaya *main thread* yang menggambar perubahan pada GUI tidak nge-*freeze*. Algoritma penyelesaian diletakkan di *thread* baru dan memberi tahu *main thread* jika butuh melanjutkan visualisasi. Setiap kali memberikan *state* dari papan saat ini, *thread* algoritma menunggu 1 ms terlebih dahulu supaya GUI terlihat ter-*update* telebih dahulu.

Fitur *live update* adalah fitur tersulit yang dibuat pada tugas ini.

D. Hasil Kodingan

1. App.java

```
package stima;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;

import java.io.IOException;
import java.util.ArrayList;
import java.util.Scanner;

public class App extends Application {
    public static App Instance;
    private static Scene scene;

    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader fxmlLoader = new
FXMLLoader(App.class.getResource("/fxml/index.fxml"));
        Parent root = fxmlLoader.load();
        scene = new Scene(root, 1080, 720);
        stage.setTitle("Permainan Queens LinkedIn");
        stage.setScene(scene);
        stage.show();
    }

    public static void setRoot(String fxml) throws IOException {
        scene.setRoot(loadFXML(fxml));
    }

    private static Parent loadFXML(String fxml) throws IOException {
        FXMLLoader fxmlLoader = new FXMLLoader(App.class.getResource(fxml +
".fxml"));
        return fxmlLoader.load();
    }

    public static void main(String[] args) {
        launch();
    }

    public static void startFromTerminal() {
        Scanner scanner = new Scanner(System.in);
        ArrayList<String> inputs = new ArrayList<String>();
        while (scanner.hasNext()) {
```

```
            String input = scanner.nextLine();
            inputs.add(input);
        }
        scanner.close();

        int n = inputs.size();
        char[][] inputGrids = new char[n][];
        for (int i = 0; i < n; i++) {
            inputGrids[i] = inputs.get(i).toCharArray();
        }

        System.out.println(Validation.validateInput(inputGrids));
        Solver.Solution solution = Solver.solve(inputGrids);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.out.print(solution.grid[i][j]);
            }
            System.out.println();
        }
        System.out.println("Total case: " + solution.totalCase);
        System.out.println("Execution time (ms): " + solution.time_ms);
    }
}
```

2. IndexController.java

```
package stima;

import java.awt.image.BufferedImage;
import java.io.File;
import java.io.IOException;
import java.io.PrintStream;
import java.net.URL;
import java.util.ArrayList;
import java.util.Scanner;

import javax.imageio.ImageIO;
import javax.swing.filechooser.FileSystemView;

import javafx.application.Platform;
import javafx.beans.value.ChangeListener;
import javafx.beans.value.ObservableValue;
import javafx.event.Event;
import javafx.fxml.FXML;
import javafx.scene.Node;
import javafx.scene.canvas.Canvas;
import javafx.scene.canvas.GraphicsContext;
import javafx.scene.control.Button;
import javafx.scene.control.CheckBox;
import javafx.scene.control.Label;
```

```java
import javafx.scene.control.TextArea;
import javafx.scene.image.Image;
import javafx.scene.image.PixelReader;
import javafx.scene.image.WritableImage;
import javafx.scene.paint.Color;
import javafx.stage.FileChooser;
import javafx.stage.Stage;
import javafx.stage.FileChooser.ExtensionFilter;
import stima.Solver.Solution;
import stima.Solver.VisualizationOption;
import stima.Validation.ValidationResult;

public class IndexController {
    public static IndexController instance;

    @FXML
    Canvas visualCanvas;
    private GraphicsContext visualContext;
    private double borderWidth = 2;

    // inputs
    @FXML
    TextArea inputTextArea;
    @FXML
    Label inputError;
    @FXML
    Button inputTxt;

    // outputs
    @FXML
    TextArea outputTextArea;
    @FXML
    Label outputError;

    // grids and solving
    @FXML
    CheckBox useVisualization;
    @FXML
    Label totalIteration;
    @FXML
    Label totalCase;

    private Image crownImage;
    private Color[] gameColors = {
            Color.web("#bba3e2"),
            Color.web("#ffc992"),
            Color.web("#96beff"),
            Color.web("#b3dfa0"),
```

```
            Color.web("#dfdfdf"),
            Color.web("#ff7b60"),
            Color.web("#e6f388"),
            Color.web("#b9b29e"),
            Color.web("#dfa0bf"),
            Color.web("#a3d2d8"),
            Color.web("#62efea"),
            Color.web("#ff93f3"),
            Color.web("#8acc6d"),
            Color.web("#729aec"),
            Color.web("#c387e0"),
            Color.web("#ffe04b"),
            Color.web("#a64210"),
            Color.web("#F4A900"),
            Color.web("#7033f2"),
            Color.web("#497E76"),
            Color.web("#ff6666"),
            Color.web("#66ff66"),
            Color.web("#ffff66"),
            Color.web("#ff66ff"),
            Color.web("#66ffff"),
            Color.web("#6666ff"),
    };

    private String defaultInput = "" +
            "AAABBCCCD¥n" +
            "ABBBBCECD¥n" +
            "ABBBDCECD¥n" +
            "AAABDCCCD¥n" +
            "BBBBDDDDD¥n" +
            "FGGGDDHDD¥n" +
            "FGIGDDHDD¥n" +
            "FGIGDDHDD¥n" +
            "FGGGDDHHH";

    private boolean isSolved = false;
    public boolean isOnLiveUpdate = false;
    public Thread liveUpdateThread = null;

    public void initialize() {
        instance = this;
        URL _url = App.class.getResource("/images/crown.png");
        crownImage = new Image(_url.toExternalForm());
        visualContext = visualCanvas.getGraphicsContext2D();

        inputTextArea.setText(defaultInput);
        inputTextArea.textProperty().addListener(new ChangeListener<String>()
{
```

```java
            @Override
            public void changed(final ObservableValue<? extends String>
observable, final String oldValue,
                    final String newValue) {
                updateGridFromInput();
            }
        });

        outputTextArea.setEditable(false);

        updateGridFromInput();
    }

    private char[][] getInput() {
        String inputString = inputTextArea.getText();
        Scanner scanner = new Scanner(inputString);

        ArrayList<String> inputs = new ArrayList<String>();
        while (scanner.hasNext()) {
            String input = scanner.nextLine();
            inputs.add(input);
        }
        scanner.close();

        int n = inputs.size();
        char[][] inputGrids = new char[n][];
        for (int i = 0; i < n; i++) {
            inputGrids[i] = inputs.get(i).toCharArray();
        }

        return inputGrids;
    }

    public void updateGridFromInput() {
        if (isOnLiveUpdate) {
            inputError.setText("Visualisasi sedang berjalan! Tunggu sampai
selesai.");
            return;
        }
        inputError.setText("");
        outputError.setText("");

        isSolved = false;

        char[][] inputGrids = getInput();
        int n = inputGrids.length;
```

```
        ValidationResult validationResult =
Validation.validateInput(inputGrids);
        switch (validationResult) {
            case Valid:
                inputError.setText("");
                outputError.setText("");
                setTotalCase(-1);
                setTotalIteration(-1);
                drawBoard(inputGrids);
                break;

            case InconsistentDimension:
                inputError.setText("Banyak kolom tidak konsisten dengan banyak
baris!");
                break;

            case WrongNumberOfRegion:
                inputError.setText(
                        "Total daerah kurang dari " + n + "! Pastikan ada
daerah dengan kode dari 'A' sampai 'Z'.");
                break;

            case InvalidColorCode:
                inputError.setText(
                        "Kode daerah tidak valid! Pastikan hanya ada daerah
dengan kode dari 'A' sampai 'Z'.");
                break;

            case SeparatedRegion:
                inputError.setText(
                        "Daerah tidak boleh terpisah!");
                break;

            case TooBig:
                inputError.setText(
                        "Besar papan maksimal adalah " +
Validation.maxGridSize + " 26.");
                break;

            case ZeroSize:
                inputError.setText(
                        "Besar papan harus lebih dari 0.");
                break;

            default:
                break;
        }
```

```
        outputTextArea.setText("");
    }

    public void onInputFromFile(Event event) {
        if (isOnLiveUpdate) {
            inputError.setText("Visualisasi sedang berjalan! Tunggu sampai
selesai.");
            return;
        }
        inputError.setText("");
        outputError.setText("");

        Node source = (Node) event.getSource();
        Stage stage = (Stage) source.getScene().getWindow();

        FileChooser fileChooser = new FileChooser();
        fileChooser.setInitialDirectory(FileSystemView.getFileSystemView().get
DefaultDirectory());
        fileChooser.setSelectedExtensionFilter(new ExtensionFilter("TEXT files
(*.txt)", "*.txt"));
        fileChooser.setTitle("Open .txt");
        File file = fileChooser.showOpenDialog(stage);

        if (file != null) {
            try {
                Scanner sc = new Scanner(file);
                String inputString = "";
                boolean first = true;
                while (sc.hasNextLine()) {
                    if (first)
                        inputString += sc.nextLine();
                    else
                        inputString += "¥n" + sc.nextLine();
                    first = false;
                }
                inputTextArea.setText(inputString);
                sc.close();

                updateGridFromInput();
            } catch (IOException e) {
                inputError.setText(e.getMessage());
            }
        }
    }

    public void onSaveAsTxt(Event event) {
        if (isOnLiveUpdate) {
```

```
            outputError.setText("Visualisasi sedang berjalan! Tunggu sampai
selesai.");
            return;
        }
        if (!isSolved) {
            outputError.setText("Selesaikan suatu papan terlebih dahulu untuk
menyimpan hasil.");
            return;
        }
        inputError.setText("");
        outputError.setText("");

        Node source = (Node) event.getSource();
        Stage stage = (Stage) source.getScene().getWindow();

        FileChooser fileChooser = new FileChooser();
        fileChooser.setInitialDirectory(FileSystemView.getFileSystemView().get
DefaultDirectory());
        fileChooser.setInitialFileName("solution.txt");
        fileChooser.setSelectedExtensionFilter(new ExtensionFilter("TEXT files
(*.txt)", "*.txt"));
        fileChooser.setTitle("Save as .txt");
        File file = fileChooser.showSaveDialog(stage);

        if (file != null) {
            try {
                PrintStream writer = new PrintStream(file);
                writer.print(outputTextArea.getText());
                writer.close();
            } catch (IOException e) {
                outputError.setText(e.getMessage());
            }
        }
    }

    public void onSaveAsPng(Event event) {
        if (isOnLiveUpdate) {
            outputError.setText("Visualisasi sedang berjalan! Tunggu sampai
selesai.");
            return;
        }
        if (!isSolved) {
            outputError.setText("Selesaikan suatu papan terlebih dahulu untuk
menyimpan hasil.");
            return;
        }
        inputError.setText("");
        outputError.setText("");
```

```
        Node source = (Node) event.getSource();
        Stage stage = (Stage) source.getScene().getWindow();

        FileChooser fileChooser = new FileChooser();
        fileChooser.setInitialDirectory(FileSystemView.getFileSystemView().get
DefaultDirectory());
        fileChooser.setInitialFileName("solution.png");
        fileChooser.setSelectedExtensionFilter(new ExtensionFilter("IMAGES
files (*.png)", "*.png"));
        fileChooser.setTitle("Save as .png");
        File file = fileChooser.showSaveDialog(stage);

        if (file != null) {
            try {
                int width = (int) visualCanvas.getWidth();
                int height = (int) visualCanvas.getHeight();
                WritableImage writableImage = new WritableImage(width, height);
                visualCanvas.snapshot(null, writableImage);
                PixelReader pixelReader = writableImage.getPixelReader();
                BufferedImage bufferedImage = new BufferedImage(width, height,
BufferedImage.TYPE_INT_ARGB);
                for (int y = 0; y < height; y++) {
                    for (int x = 0; x < width; x++) {
                        int argb = pixelReader.getArgb(x, y);
                        bufferedImage.setRGB(x, y, argb);
                    }
                }
                ImageIO.write(bufferedImage, "png", file);
            } catch (IOException e) {
                outputError.setText(e.getMessage());
            }
        }
    }

    public void solveGrid() {
        if (isOnLiveUpdate) {
            outputError.setText("Visualisasi sedang berjalan! Tunggu sampai
selesai.");
            return;
        }
        if (Validation.validateInput(getInput()) != ValidationResult.Valid) {
            return;
        }
        inputError.setText("");
        outputError.setText("");
        outputTextArea.setText("");
```

```java
        Solution solution;
        char[][] inputGrids = getInput();
        if (useVisualization.isSelected()) {
            Solver.solve(inputGrids, VisualizationOption.Live);
        } else {
            solution = Solver.solve(inputGrids, VisualizationOption.None);
            outputSolution(solution);
        }
    }

    public void outputSolution(Solution solution) {
        isSolved = true;

        String outputString = new String();
        if (solution.isSolved) {
            int n = solution.grid.length;
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    outputString += solution.grid[i][j];
                }
                outputString += '¥n';
            }
            outputString += '¥n';
            outputString += "Solution found.¥n";
        } else {
            outputString += "Solution not found.¥n";
        }
        outputString += "Execution time: " + solution.time_ms + " ms¥n";
        outputString += "Total iteration: " + solution.totalIteration + "¥n";
        outputString += "Total case: " + solution.totalCase + "¥n";

        outputTextArea.setText(outputString);
    }

    public void resetGrid() {
        updateGridFromInput();
    }

    public void drawBoard() {
        Platform.runLater(new Runnable() {
            @Override
            public void run() {
                double width = visualCanvas.getWidth();
                double height = visualCanvas.getHeight();
                visualContext.setFill(Color.BLACK);
                visualContext.fillRect(0, 0, width, height);
                double cellWidth = (width - 6 * borderWidth) / 5.0;
                for (int i = 0; i < 5; i++) {
```

```
                for (int j = 0; j < 5; j++) {
                    visualContext.setFill(Color.WHITE);
                    visualContext.fillRect(
                            borderWidth * (j + 1) + cellWidth * j,
                            borderWidth * (i + 1) + cellWidth * i,
                            cellWidth,
                            cellWidth);
                }
            }
        }
    });
}

public void drawBoard(char[][] input) {
    Platform.runLater(new Runnable() {
        @Override
        public void run() {
            double width = visualCanvas.getWidth();
            double height = visualCanvas.getHeight();
            visualContext.setFill(Color.BLACK);
            visualContext.fillRect(0, 0, width, height);
            double n = input.length;
            double cellWidth = (width - (n + 1) * borderWidth) / n;
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    visualContext.setFill(gameColors[input[i][j] - 'A']);
                    visualContext.fillRect(
                            borderWidth * (j + 1) + cellWidth * j,
                            borderWidth * (i + 1) + cellWidth * i,
                            cellWidth,
                            cellWidth);
                }
            }
        }
    });
}

public void drawBoard(char[][] input, char[][] output) {
    Platform.runLater(new Runnable() {
        @Override
        public void run() {
            double width = visualCanvas.getWidth();
            double n = input.length;
            double cellWidth = (width - (n + 1) * borderWidth) / n;

            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    visualContext.setFill(gameColors[input[i][j] - 'A']);
```

```
                        visualContext.fillRect(
                                borderWidth * (j + 1) + cellWidth * j,
                                borderWidth * (i + 1) + cellWidth * i,
                                cellWidth,
                                cellWidth);

                        if (output[i][j] == '#') {
                            double queenWidth = cellWidth / 2;
                            visualContext.setFill(Color.BLACK);
                            visualContext.drawImage(crownImage,
                                    borderWidth * (j + 1) + cellWidth * j +
queenWidth / 2,
                                    borderWidth * (i + 1) + cellWidth * i +
queenWidth / 2,
                                    queenWidth,
                                    queenWidth);
                        }
                    }
                }
            }
        });
    }

    public void drawCell(int i, int j, int n, char code) {
        Platform.runLater(new Runnable() {
            @Override
            public void run() {
                double width = visualCanvas.getWidth();
                double cellWidth = (width - (n + 1) * borderWidth) / n;

                if (code == '#') {
                    double queenWidth = cellWidth / 2;
                    visualContext.setFill(Color.BLACK);
                    visualContext.drawImage(crownImage,
                            borderWidth * (j + 1) + cellWidth * j + queenWidth
/ 2,
                            borderWidth * (i + 1) + cellWidth * i + queenWidth
/ 2,
                            queenWidth,
                            queenWidth);
                } else {
                    visualContext.setFill(gameColors[code - 'A']);
                    visualContext.fillRect(
                            borderWidth * (j + 1) + cellWidth * j,
                            borderWidth * (i + 1) + cellWidth * i,
                            cellWidth,
                            cellWidth);
                }
```

```
            }
        });
    }

    public void setTotalIteration(int n) {
        Platform.runLater(new Runnable() {
            @Override
            public void run() {
                totalIteration.setText("Total iteration: " + (n == -1 ? "-" :
n));
            }
        });
    }

    public synchronized void setTotalCase(int n) {
        Platform.runLater(new Runnable() {
            @Override
            public void run() {
                totalCase.setText("Total case: " + (n == -1 ? "-" : n));
            }
        });
    }
}
```

3. Solver.java

```
package stima;

import java.util.concurrent.TimeUnit;

public class Solver {
    public static class Solution {
        boolean isSolved;
        char[][] grid;
        int totalCase;
        int totalIteration;
        long time_ms;
    }

    public static enum VisualizationOption {
        Live,
        None
    }

    private static class StatCount {
        int totalCase;
        int totalIteration;
```

```java
        StatCount() {
            this.totalCase = 0;
            this.totalIteration = 0;
        }
    }

    public static Solution solve(char[][] input) {
        return solve(input, VisualizationOption.None);
    }

    public static Solution solve(char[][] input, VisualizationOption option) {
        // Thread for visualization
        // Biar GUI nya ga ngefreeze
        IndexController.instance.drawBoard(input);
        IndexController.instance.setTotalCase(0);
        IndexController.instance.setTotalIteration(0);
        if (option == VisualizationOption.Live) {
            IndexController.instance.liveUpdateThread = new Thread(() -> {
                IndexController.instance.isOnLiveUpdate = true;
                long startTime = System.nanoTime();
                int n = input.length;
                boolean[] usedColor = new boolean[26];
                int[] usedColumn = new int[n];
                for (int i = 0; i < n; i++) {
                    usedColor[i] = false;
                    usedColumn[i] = -1;
                }
                Solution solution = solveRecursive(input, 0, usedColor,
usedColumn, option, new StatCount());

                long endTime = System.nanoTime();
                solution.time_ms = TimeUnit.NANOSECONDS.toMillis(endTime -
startTime);

                IndexController.instance.outputSolution(solution);
                IndexController.instance.isOnLiveUpdate = false;
            });
            IndexController.instance.liveUpdateThread.start();
            return null;
        } else {
            long startTime = System.nanoTime();
            int n = input.length;
            boolean[] usedColor = new boolean[26];
            int[] usedColumn = new int[n];
            for (int i = 0; i < n; i++) {
                usedColor[i] = false;
                usedColumn[i] = -1;
```

```
            }

            Solution solution = solveRecursive(input, 0, usedColor, usedColumn,
VisualizationOption.None,
                    new StatCount());

            long endTime = System.nanoTime();
            solution.time_ms = TimeUnit.NANOSECONDS.toMillis(endTime -
startTime);

            if (option == VisualizationOption.None) {
                tryUpdateGUI(() -> {
                    if (solution.isSolved)
                        IndexController.instance.drawBoard(input,
solution.grid);
                    IndexController.instance.setTotalIteration(solution.totalI
teration);
                    IndexController.instance.setTotalCase(solution.totalCase);
                });
            }

            return solution;
        }
    }

    private static Solution solveRecursive(char[][] input, int i, boolean[]
usedColor, int[] usedColumn,
            VisualizationOption option, StatCount statCount) {
        int n = input.length;
        boolean hasSolution = false;
        for (int j = 0; j < n; j++) {
            statCount.totalIteration++;
            if (!checkValidity(input, i, j, usedColor, usedColumn)) {
                statCount.totalCase++;
                if (option != VisualizationOption.None) {
                    int temp = j;
                    tryUpdateGUI(() -> {
                        IndexController.instance.drawCell(i, temp,
input.length, '#');
                        IndexController.instance.setTotalCase(statCount.totalC
ase);
                        IndexController.instance.setTotalIteration(statCount.t
otalIteration);
                    });
                    tryUpdateGUI(() -> {
                        IndexController.instance.drawCell(i, temp,
input.length, input[i][temp]);
                    });
```

```
                }
                continue;
            }

            // Tandai kolom dan warna yang dipakai
            addQueen(input, i, j, usedColor, usedColumn, option, statCount);

            // Sudah sampai ujung, pasti ini solusinya
            if (i == n - 1) {
                hasSolution = true;
                statCount.totalCase++;
                break;
            }

            // Lanjut ke baris selanjutnya
            Solution solution = solveRecursive(input, i + 1, usedColor,
usedColumn, option, statCount);

            if (solution.isSolved) { // Dapat solusi
                return solution;
            } else { // Tidak dapat solusi, balikkan state seperti semula
                removeQueen(input, i, j, usedColor, usedColumn, option,
statCount);
            }
        }

        Solution solution = new Solution();
        solution.totalCase = statCount.totalCase;
        solution.totalIteration = statCount.totalIteration;
        solution.isSolved = hasSolution;
        if (hasSolution) {
            solution.grid = generateSolutionGrid(input, usedColumn);
        }
        return solution;
    }

    private static boolean checkValidity(char[][] input, int i, int j,
boolean[] usedColor, int[] usedColumn) {
        int n = input.length;

        // Kalau kolom ini sudah dipakai
        if (usedColumn[j] != -1) {
            return false;
        }
        // Kalau warna ini sudah dipakai
        if (usedColor[input[i][j] - 'A']) {
            return false;
        }
```

```
        // Kalau "queen" sebelumnya berdekatan (cukup cek diagonal karena
kolom sudah
        // dicek)
        if (i > 0 && ((j > 0 && usedColumn[j - 1] == i - 1) || (j < n - 1 &&
usedColumn[j + 1] == i - 1))) {
            return false;
        }
        return true;
    }

    private static char[][] generateSolutionGrid(char[][] input, int[]
usedColumn) {
        int n = input.length;
        char[][] solutionGrid = new char[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (usedColumn[j] == i) {
                    solutionGrid[i][j] = '#';
                } else {
                    solutionGrid[i][j] = input[i][j];
                }
            }
        }
        return solutionGrid;
    }

    private static void addQueen(char[][] input, int i, int j, boolean[]
usedColor, int[] usedColumn,
            VisualizationOption option, StatCount statCount) {
        usedColor[input[i][j] - 'A'] = true;
        usedColumn[j] = i;
        if (option != VisualizationOption.None) {
            tryUpdateGUI(() -> {
                IndexController.instance.drawCell(i, j, input.length, '#');
                IndexController.instance.setTotalCase(statCount.totalCase);
                IndexController.instance.setTotalIteration(statCount.totalIter
ation);
            });
        }
    }

    private static void removeQueen(char[][] input, int i, int j, boolean[]
usedColor, int[] usedColumn,
            VisualizationOption option, StatCount statCount) {
        usedColor[input[i][j] - 'A'] = false;
        usedColumn[j] = -1;
        if (option != VisualizationOption.None) {
            tryUpdateGUI(() -> {
```

```
                IndexController.instance.drawCell(i, j, input.length,
input[i][j]);
                IndexController.instance.setTotalCase(statCount.totalCase);
                IndexController.instance.setTotalIteration(statCount.totalIter
ation);
            });
        }
    }

    private static void tryUpdateGUI(Runnable task) {
        try {
            task.run();
            Thread.sleep(1);
        } catch (Exception e) {
            System.err.println(e.getMessage());
            System.err.println(e.getStackTrace());
        }
    }
}
```

4. Validation.java

```
package stima;

import java.util.Arrays;

public class Validation {
    public static final int maxGridSize = 26;

    public static enum ValidationResult {
        Valid,
        InconsistentDimension,
        InvalidColorCode,
        WrongNumberOfRegion,
        SeparatedRegion,
        TooBig,
        ZeroSize
    }

    public static ValidationResult validateInput(char[][] input) {
        int n = input.length;
        for (char[] cs : input) {
            if (cs.length != n)
                return ValidationResult.InconsistentDimension;
        }

        if (n == 0)
            return ValidationResult.ZeroSize;
```

```java
        if (n > maxGridSize) {
            return ValidationResult.TooBig;
        }

        int totalRegion = 0;
        boolean[] hasColor = new boolean[26];
        boolean[][] isAdjacent = new boolean[n][n];
        Arrays.fill(hasColor, false);

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (input[i][j] < 'A' || input[i][j] > 'Z')
                    return ValidationResult.InvalidColorCode;
                if (hasColor[input[i][j] - 'A']) {
                    if (!isAdjacent[i][j])
                        return ValidationResult.SeparatedRegion;
                } else {
                    hasColor[input[i][j] - 'A'] = true;
                    dfsAdjacentColor(i, j, isAdjacent, input);
                    totalRegion++;
                }
            }
        }
        if (totalRegion != n)
            return ValidationResult.WrongNumberOfRegion;
        return ValidationResult.Valid;
    }

    public static void dfsAdjacentColor(int i, int j, boolean[][] isAdjacent,
char[][] input) {
        isAdjacent[i][j] = true;
        int n = isAdjacent.length;
        int[] offsetI = { 0, -1, 0, 1 };
        int[] offsetJ = { -1, 0, 1, 0 };
        for (int k = 0; k < 4; k++) {
            int new_i = i + offsetI[k];
            int new_j = j + offsetJ[k];
            if (new_i < 0 || new_j < 0 || new_i >= n || new_j >= n)
                continue;
            if (input[i][j] != input[new_i][new_j] || isAdjacent[new_i][new_j])
                continue;
            dfsAdjacentColor(new_i, new_j, isAdjacent, input);
        }
    }
}
```

Hasil Pengujian

Kasus uji diambil dari dokumen spesifikasi tugas dan permainan Queens dari komunitas (lihat lampiran untuk link).

A. Kasus uji 1

Input:

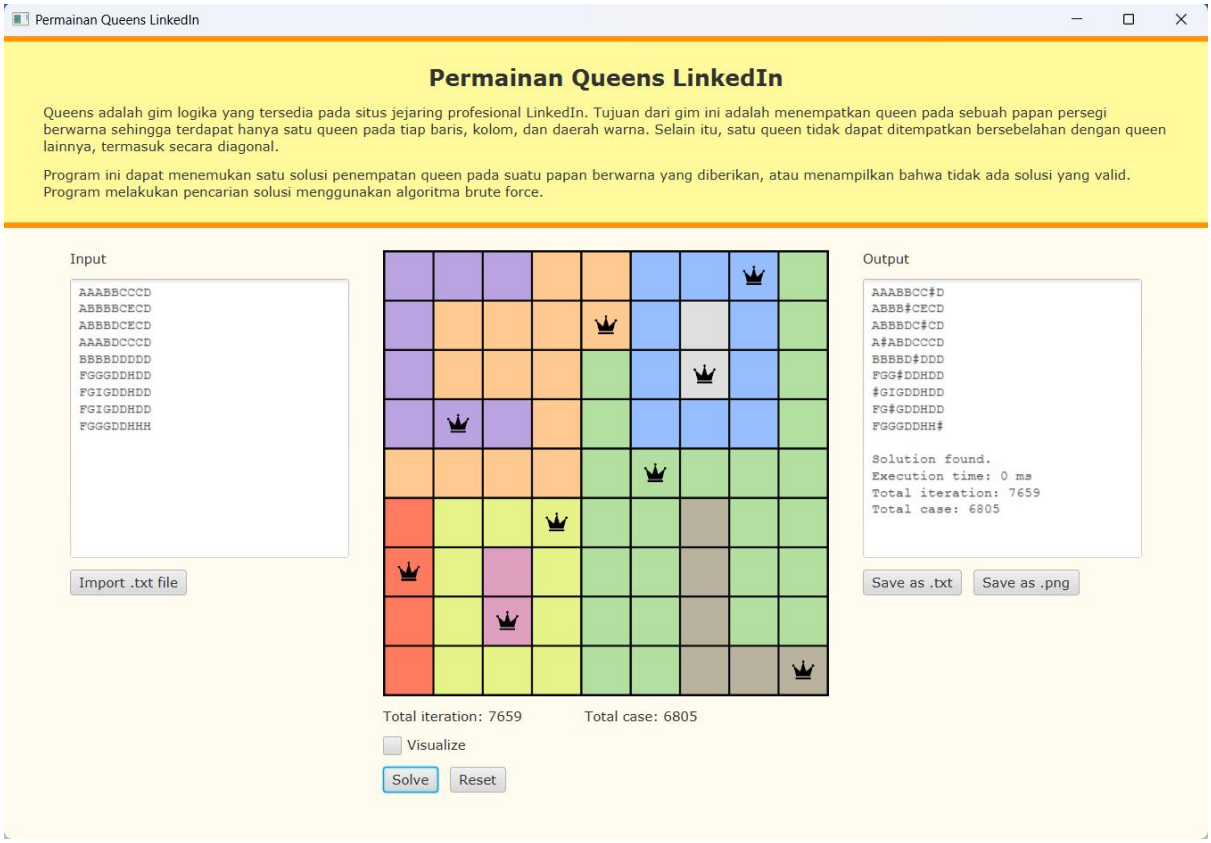
```
AAABBCCCD
ABBBBCECD
ABBBDCECD
AAABDCCCD
BBBBDDDDD
FGGGDDHDD
FGIGDDHDD
FGIGDDHDD
FGGGDDHHH
```

Output:

```
AAABBCC#D
ABBB#CECD
ABBBDC#CD
A#ABDCCCD
BBBBD#DDD
FGG#DDHDD
#GIGDDHDD
FG#GDDHDD
FGGGDDHH#

Solution found.
Execution time: 0 ms
Total iteration: 7659
Total case: 6805
```

Tampilan:



B. Kasus uji 2

Input:

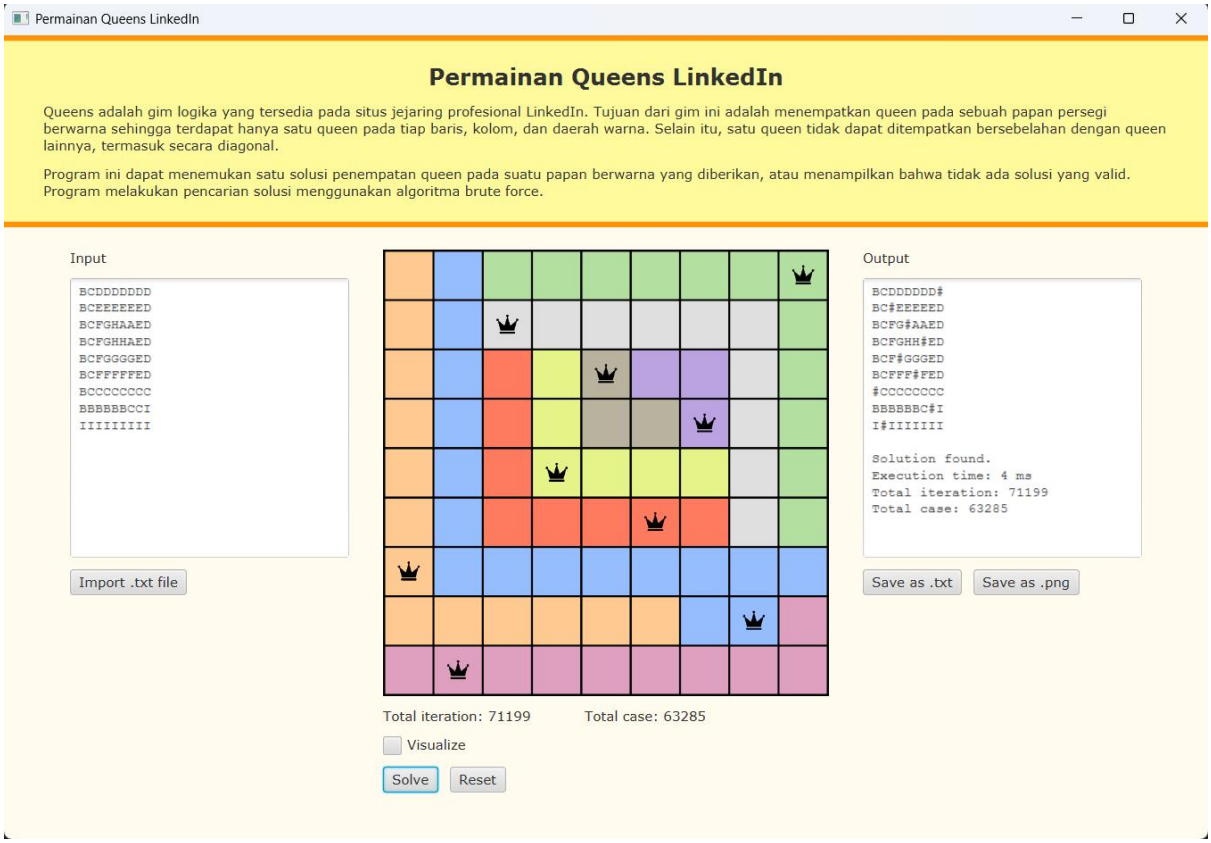
```
BCDDDDDDD
BCEEEEEED
BCFGHAAED
BCFGHHAED
BCFGGGGED
BCFFFFFED
BCCCCCCCC
BBBBBBCCI
IIIIIIIII
```

Output:

```
BCDDDDDD#
BC#EEEEED
BCFG#AAED
BCFGHH#ED
BCF#GGGED
BCFFF#FED
#CCCCCCCC
BBBBBBC#I
I#IIIIIII

Solution found.
Execution time: 4 ms
Total iteration: 71199
Total case: 63285
```

Tampilan:



C. Kasus uji 3

Input:

```
KKKKKBBGGGG
KKKKBBGGBBF
KKKKKBBBBEF
IIKKKKKBEEF
IKKKABBBEFF
IKAAAJJBBBD
IAAJAJJJDDD
IAIJJJDDDDD
IIICCDDDHHH
IIIICCCDHIH
IIIIIIIIIIH
```

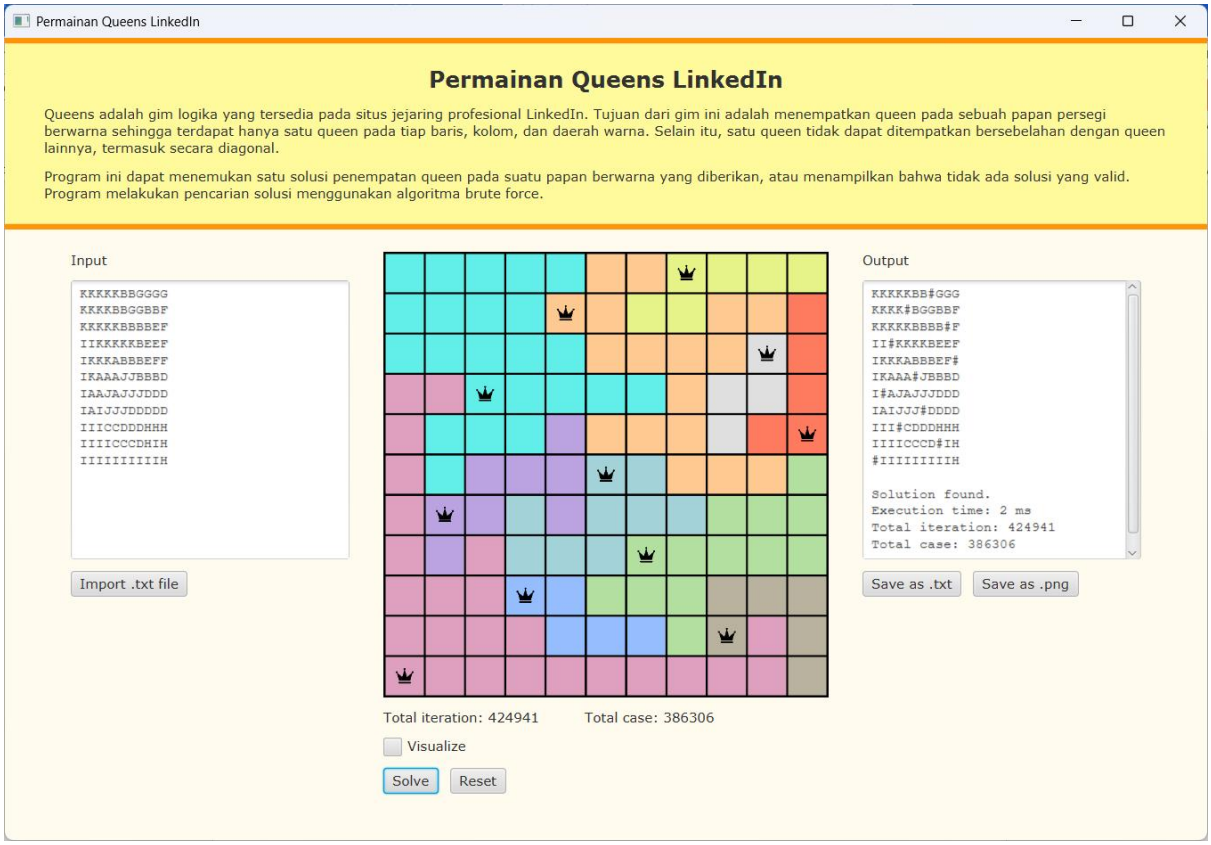
Output:

```
KKKKKBB#GGG
KKKK#BGGBBF
KKKKKBBBB#F
II#KKKKBEEF
IKKKABBBEF#
IKAAA#JBBBD
I#AJAJJJDDD
IAIJJJ#DDDD
```

```
III#CDDDHHH
IIIICCCD#IH
#IIIIIIIIIH

Solution found.
Execution time: 2 ms
Total iteration: 424941
Total case: 386306
```

Tampilan:



D. Kasus uji 4

Input:

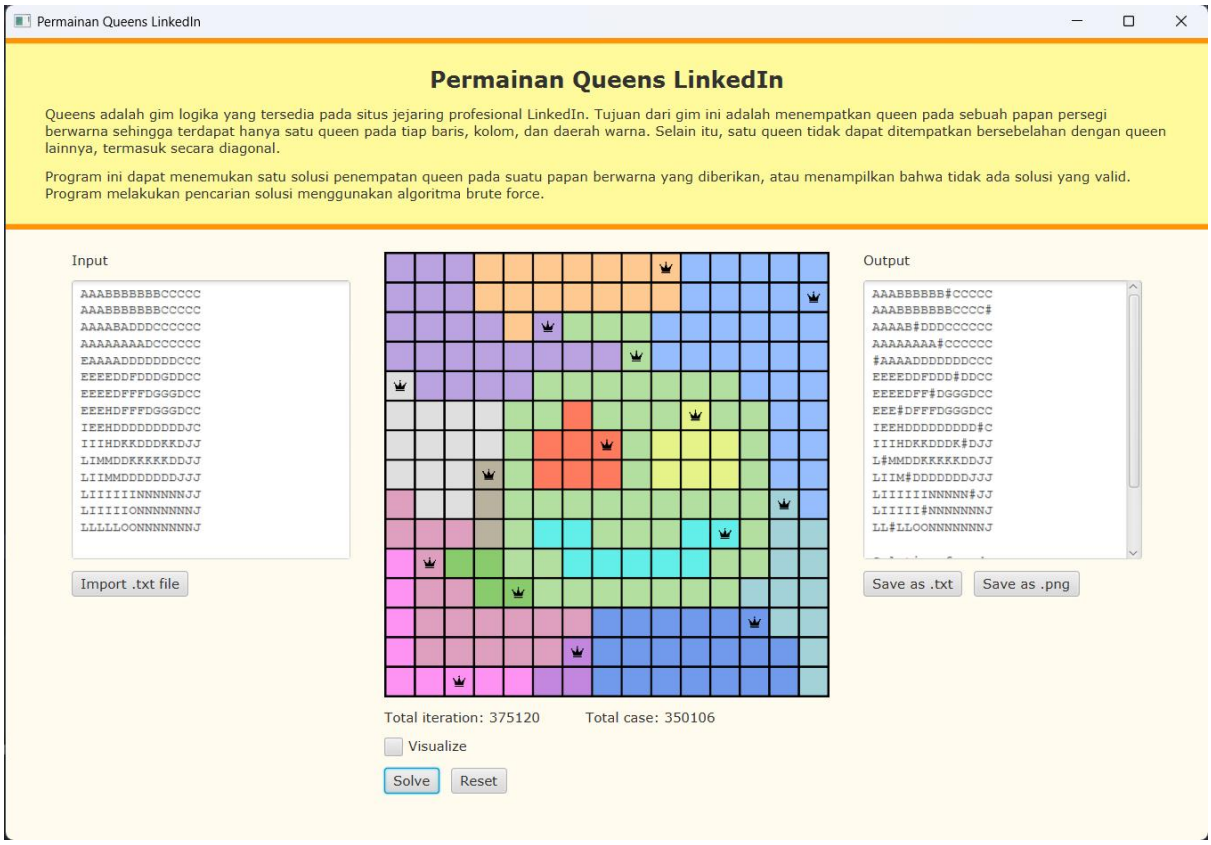
```
AAABBBBBBBCCCCC
AAABBBBBBBCCCCC
AAAABADDDCCCCCC
AAAAAAAADCCCCCC
EAAAADDDDDDDCCC
EEEEDDFDDDGDDCC
EEEEDFFFDGGGDCC
EEEHDFFFDGGGDCC
IEEHDDDDDDDDDJC
IIIHDKKDDDKKDJJ
LIMMDDKKKKKDDJJ
LIIMMDDDDDDDJJJ
LIIIIIINNNNNNJJ
LIIIIIONNNNNNNJ
LLLLLOONNNNNNNJ
```

Output:

```
AAABBBBBB#CCCCC
AAABBBBBBBCCCC#
AAAAB#DDDCCCCCC
AAAAAAAA#CCCCCC
#AAAADDDDDDDCCC
EEEEDDFDDD#DDCC
EEEEDFF#DGGGDCC
EEE#DFFFDGGGDCC
IEEHDDDDDDDDD#C
IIIHDKKDDDK#DJJ
L#MMDDKKKKKDDJJ
LIIM#DDDDDDDJJJ
LIIIIIINNNNN#JJ
LIIIII#NNNNNNNJ
LL#LLOONNNNNNNJ

Solution found.
Execution time: 2 ms
Total iteration: 375120
Total case: 350106
```

Tampilan:



E. Kasus uji 5

Input:

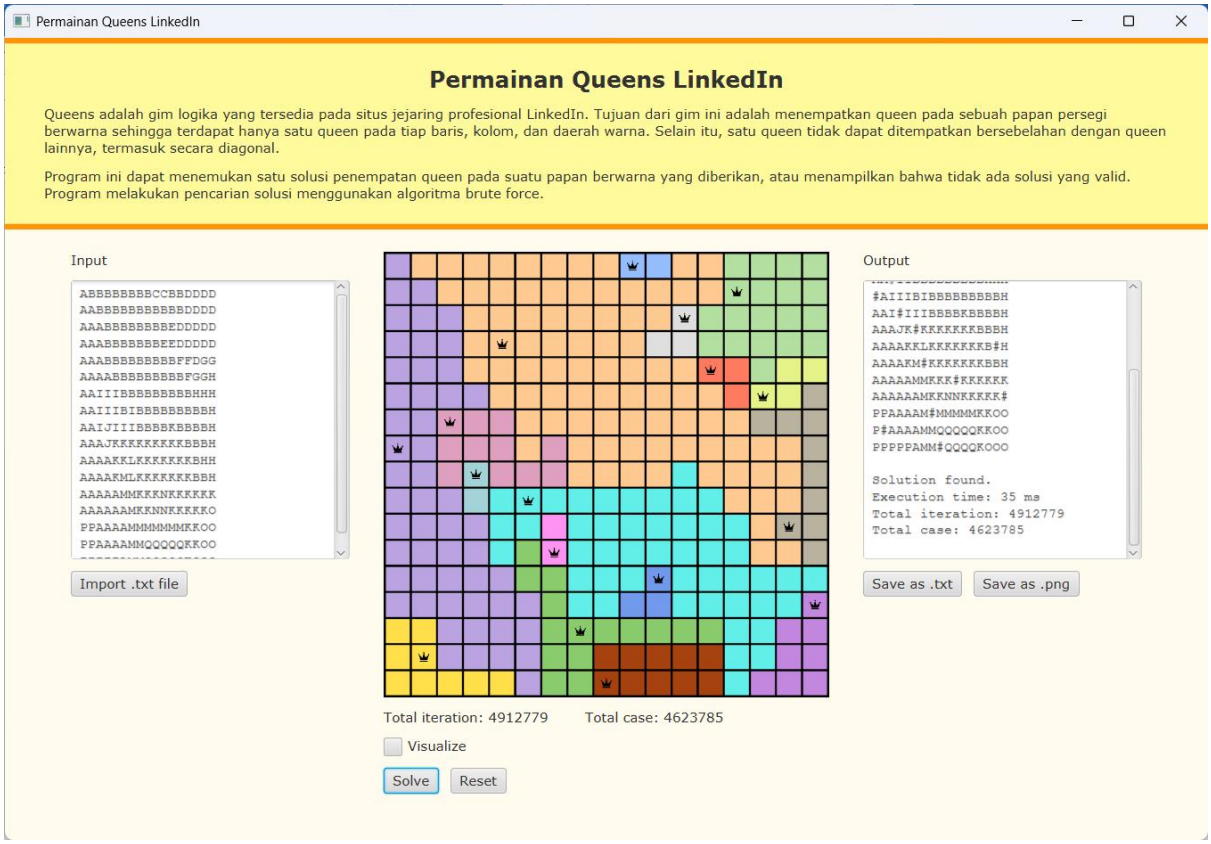
```
ABBBBBBBBCCBBDDDD
AABBBBBBBBBBBDDDD
AAABBBBBBBBEDDDDD
AAABBBBBBBEEDDDDD
AAABBBBBBBBBFFDGG
AAAABBBBBBBBBFGGH
AAIIIBBBBBBBBBHHH
AAIIIBIBBBBBBBBBH
AAIJIIIBBBBKBBBBH
AAAJKKKKKKKKKBBBH
AAAAKKLKKKKKKKBHH
AAAAKMLKKKKKKKBBH
AAAAAMMKKKNKKKKKK
AAAAAAMKKNNKKKKKO
PPAAAAMMMMMMMKKOO
PPAAAAMMQQQQQKKOO
PPPPPAMMQQQQQKOOO
```

Output:

```
ABBBBBBBB#CBBDDDD
AABBBBBBBBBBB#DDD
AAABBBBBBBB#DDDDD
AAAB#BBBBBEEDDDDD
AAABBBBBBBBB#FDGG
AAAABBBBBBBBBF#GH
AA#IIBBBBBBBBBHHH
#AIIIBIBBBBBBBBBH
AAI#IIIBBBBKBBBBH
AAAJK#KKKKKKKBBBH
AAAAKKLKKKKKKKB#H
AAAAKM#KKKKKKKBBH
AAAAAMMKKK#KKKKKK
AAAAAAMKKNNKKKKK#
PPAAAAM#MMMMMKKOO
P#AAAAMMQQQQQKKOO
PPPPPAMM#QQQQKOOO

Solution found.
Execution time: 35 ms
Total iteration: 4912779
Total case: 4623785
```

Tampilan:



Penutup

Solusi dapat ditemukan dengan sangat cepat untuk besar papan yang kecil (n < 20). Jadi kurang terasa *brute force* nya. Visualisasi sudah cukup bagus dalam membuat rasa *brute force* muncul. Perlu dilakukan percobaan untuk algoritma yang lebih lama, yaitu membuat posisi akhir dari *queen* terlebih dahulu, baru mengecek kevalidannya.

Terima kasih teman-teman yang telah berjuang dengan saya di tugas kecil ini :D

Lampiran

Tabel *checklist* poin-poin pada program

No.	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan	V	X
2	Program berhasil dijalankan	V	X
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	V	X
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	V	X
5	Program memiliki Graphical User Interface (GUI)	V	X
6	Program dapat menyimpan solusi dalam bentuk file gambar	V	X

Pernyataan penggunaan AI:

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Muhammad Nafis Habibi

Link Github: https://github.com/NafisKreatif/Tucil1_13524018

Link level komunitas: https://queensgame.vercel.app/community-level